

Felvételi, 2024. július
Szoftverfejlesztés Mesterképzés
Javítókulcs

Programozás és programozási nyelvek

```
void updateLeds( int *leds, char *pattern){
    for( int i = 0 ; i < 7 ; ++i ){
        leds[i] += pattern[i] - '0';
    }
}

char mostDemandedSegment( int n, int *a ){

    //segment demands
    int leds[7] = {0};

    // LED-patterns for digits 0,1,2,3,4,5,6,7,8,9
    char *patterns[] = {"111110", "0110000", "1101101", "1111001", "0110011",
                        "1011011", "1011111", "1110000", "1111111", "1111011"};

    //for every element of the sequence
    for( int i = 0 ; i < n ; ++i ){

        if( a[i] == 0 ){ updateLeds(leds,patterns[0]); continue; }

        if( a[i] < 0 ){ a[i] = -a[i]; }

        //for every digit of the current element
        while( a[i] ){
            updateLeds(leds,patterns[a[i]%10]);
            a[i] /= 10;
        }
    }

    //identifying the most demanded segment
    int minpos = 0;
    for( int i = 1 ; i < 7 ; ++i ){
        if( leds[i] > leds[minpos] ){ minpos = i; }
    }

    return minpos + 'a';
}
```

Adatbázisok

1.

```
select head_coach
from countries
where country in ( select country
                  from jumpers
                  group by country
                  having count(*)= (select max(count(*))
                                   from jumpers
                                   group by country));
```

2.

```
select hill_name
from hills
where hill_record > hill_size * 1.07;
```

3.

```
select sum(point) - min(point) - max(point) as jury_points
from evaluations
where j.jump_id = 10001;
```

4.

```
select tj.jumper_name, e.jury_points + j.distance * 1.6 as total_points
from jumps j
join (
    select jump_id, sum(point) - min(point) - max(point) as jury_points
    from evaluations
    group by jump_id
) e
on j.jump_id = e.jump_id
join jumpers tj on j.jumper_id = tj.jumper_id
where j.comp_id = 1
order by total_points desc;
```

5.

```
with JumperAverages as (
    select j.jumper_id, j.jumper_name, c.country, avg(jj.distance) as
average_distance
    from jumpers j join countries c
    on j.country = c.country
    join jumps jj
    on j.jumper_id = jj.jumper_id
```

```
        group by j.jumper_id, j.jumper_name, c.country)
select country, jumper_name, round(average_distance,2)
from JumperAverages a
where (country, average_distance) in (
        select country, max(average_distance) as max_distance
        from JumperAverages
        group by country
    )
order by average_distance desc;
```

7.

```
select j.jumper_name
from sponsorship s
join jumpers j
on j.jumper_id = s.jumper_id
group by j.jumper_name
having sum(s.amount) >= 6000
order by j.birthdate desc
fetch first row only;
```

8.

```
select j.jumper_name, count(*)
from jumpers j join jumps jj
on j.jumper_id = jj.jumper_id
group by j.jumper_name, jj.comp_id
having count(*)>2;
```

9.

```
select j.jumper_name, extract(year from c.comp_date) - extract(year from j.birthdate) as
yearold
from jumpers j
join jumps jj on j.jumper_id = jj.jumper_id
join competitions c on jj.comp_id = c.comp_id
where extract(month from j.birthdate) = extract(month from c.comp_date)
and extract(day from j.birthdate) = extract(day from c.comp_date)
and extract(year from j.birthdate) != EXTRACT(year from c.comp_date);
```

Objektumelvű programozás és szoftvertervezés

1.

```
public class Tantargy {
    private String tantargyNev;
```

```
private int kredit;  
private int jegy;  
  
public Tantargy(String tantargyNev, int kredit) {  
    this.tantargyNev = tantargyNev;  
    this.kredit = kredit;  
}  
  
public void setJegy(int jegy) {  
    this.jegy = jegy;  
}  
  
public String getTantargyNev() {  
    return tantargyNev;  
}  
  
public int getKredit() {  
    return kredit;  
}  
  
public int getJegy() {  
    return jegy;  
}  
  
@Override  
public String toString() {  
    return "Tantargy{" +  
        "tantargyNev='" + tantargyNev + '\'' +  
        ", kredit=" + kredit +  
        ", jegy=" + jegy +  
        '}';  
}  
}
```

2.

```
import java.util.ArrayList;  
import java.util.List;  
  
public class Hallgato {  
    private final int ID;  
    private static int counter;  
    private String nev;  
    private List<Tantargy> tantargyak = new ArrayList<>();  
  
    public Hallgato(String nev) {  
        this.nev = nev;  
        this.ID = ++counter;  
    }  
  
    public String getNev() {
```

```
        return nev;
    }

    public void ujTantargy(Tantargy tantargy) {
        tantargyak.add(tantargy);
    }

    public void vizsgaJegy(String tantargyNev, int jegy) {
        for (Tantargy tantargy : tantargyak) {
            if (tantargy.getTantargyNev().equals(tantargyNev)) {
                tantargy.setJegy(jegy);
                return;
            }
        }
    }

    public int getJegy(String tantargyNev) {
        for (Tantargy tantargy : tantargyak) {
            if (tantargy.getTantargyNev().equals(tantargyNev)) {
                return tantargy.getJegy();
            }
        }
        throw new RuntimeException("Nem letezo tantargy!");
    }

    public int osszKredit() {
        int ossz = 0;
        for(Tantargy tantargy: tantargyak) {
            if( tantargy.getJegy() >=5 ) {
                ossz += tantargy.getJegy();
            }
        }
        return ossz;
    }

    public double atlag() {
        if (tantargyak.isEmpty()) {
            throw new RuntimeException("Nincs jegye a hallgatonak!");
        }
        double sum = 0;
        for (Tantargy tantargy : tantargyak) {
            sum += tantargy.getJegy();
        }
        return sum / tantargyak.size();
    }
}
```

3.

```
Hallgato hallgato = new Hallgato("Pal Istvan");
hallgato.ujTantargy( new Tantargy("OOP", 5));
hallgato.ujTantargy( new Tantargy("Programozas", 5));
```

Sapientia Erdélyi Magyar Tudományegyetem Marosvásárhelyi Kar

```
hallgato.ujTantargy( new Tantargy("Adatbazisok", 5));  
hallgato.vizsgaJegy("OOP", 10);  
hallgato.vizsgaJegy("Adatbazisok", 10);  
hallgato.vizsgaJegy("Programozas", 10);  
try{  
    System.out.println("Hallgato: " + hallgato.getNev() + " atlag: " + hallgato.atlag());  
} catch( RuntimeException e) {  
    e.printStackTrace();  
}
```

4.

```
import java.util.ArrayList;  
import java.util.List;  
  
public class Egyetem {  
    private List<Hallgato> hallgatok = new ArrayList<>();  
  
    public void ujHallgato(Hallgato hallgato) {  
        hallgatok.add(hallgato);  
    }  
  
    public void evismetlok() {  
        for(Hallgato hallgato: hallgatok) {  
            if( hallgato.osszKredit() < 30 ) {  
                System.out.println(hallgato);  
            }  
        }  
    }  
}
```

5. Mindkét kapcsolat egy a sokhoz tartalmazási kapcsolat.



6. Singleton magyarázat: olyan osztály, amelyből csak egyetlen példányt lehet készíteni.

```
public class SingletonEgyetem {
    private List<Hallgato> hallgatok = new ArrayList<>();

    private static SingletonEgyetem instance;

    private SingletonEgyetem() {
    }

    public static SingletonEgyetem getInstance() {
        if( instance == null) {
            instance = new SingletonEgyetem();
        }
        return instance;
    }
    // ...
}
```

Példány készítése és használata:

```
SingletonEgyetem sapientia = SingletonEgyetem.getInstance();
sapientia.evismetlok();
```

7. Nyílt-zárt elv:

A nyílt-zárt elv (Open-Closed Principle, OCP) a SOLID elvek egyike, és azt mondja ki, hogy egy szoftverkomponens legyen nyílt a kiterjesztésre, de zárt a módosításra. Ez azt jelenti, hogy a komponens működése bővíthető anélkül, hogy a meglévő kódot módosítani kellene.

Példa Java nyelven:

Tegyük fel, hogy van egy interfészünk, amely meghatározza egy alakzat (Shape) területének kiszámítását.

```
// Shape.java
public interface Shape {
    double calculateArea();
}

// Circle.java
public class Circle implements Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
```

```
    public double calculateArea() {  
        return Math.PI * radius * radius;  
    }  
}  
  
// Rectangle.java  
public class Rectangle implements Shape {  
    private double width;  
    private double height;  
  
    public Rectangle(double width, double height) {  
        this.width = width;  
        this.height = height;  
    }  
  
    @Override  
    public double calculateArea() {  
        return width * height;  
    }  
}  
  
// AreaCalculator.java  
public class AreaCalculator {  
    public double calculateTotalArea(Shape[] shapes) {  
        double totalArea = 0;  
        for (Shape shape : shapes) {  
            totalArea += shape.calculateArea();  
        }  
        return totalArea;  
    }  
}
```

Ebben a példában az `AreaCalculator` osztály kódja **zárt a módosításra** (nem kell módosítani, ha új alakzatokat adunk hozzá), viszont **nyílt a kiterjesztésre** (új alakzatokat egyszerűen hozzáadhatunk az interfész implementálásával, mint például a `Circle` és `Rectangle` osztályok).